

Problem 1:

1-a)

Set:  $x = [x_1; x_2]$

$$\dot{x}_1 = x_2$$

$$\dot{x}_2 = -k * x_1/m + u/m$$

Combine equations into matrix form to find the continuous time model:

$$\dot{x} = A_c x + B_c u$$

$$\dot{x} = [\dot{x}_1; \dot{x}_2] = A_c x + B_c u = [0, 1; -k/m, 0] * x + [0; 1/m]u = [0, 1; -k/m, 0] * [x_1; x_2] + [0; 1/m]u$$

$$A_c = [0, 1; -k/m, 0], B_c = [0; 1/m]$$

Putting in discrete time using python:

$$m, k = 1, T_s = 0.1$$

Code:

```
from scipy.signal import cont2discrete
m, k, Ts = 1.0, 1.0, 0.1

Ac = np.array([[0, 1], [-k/m, 0]])
Bc = np.array([[0], [1/m]])
Ad, Bd = cont2discrete((Ac, Bc, np.eye(2), np.zeros((2,1))), Ts,
    method='zoh')[:2]
```

$$A_d = [0.995, 0.0998; -0.0998, 0.995], B_d = [0.005; 0.0998]$$

1-b)

Rows 1,2

$$x_{k+1} = A_d x_k + B_d u_k$$

$$\Delta x_{k+1} = x_{k+2} - x_{k+1} = A_d x_{k+1} + B_d u_{k+1} - A_d x_k + B_d u_k = A_d (x_{k+1} - x_k) + B_d (u_{k+1} - u_k)$$

$$\Delta x_{k+1} = A_d \Delta x_k + B_d \Delta u_k$$

Row 4

$$u_{k+1} = u_k + \Delta u_k$$

Row 5

$$x_{1,k+1} = x_{1,k} + \Delta x_{1,k}$$

Row 3

$$e_k = x_{1,k} - r_k$$

The reference is constant at each time step, so  $r_{k+1} = r_k$ :

$$e_{k+1} = x_{1,k+1} - r_{k+1} = x_{1,k} + \Delta x_{1,k} - r_k = \Delta x_{1,k} + (x_{1,k} - r_k) = \Delta x_{1,k} + e_k$$

Putting this all together yields that augmented state space formulated given in the problem formulation.  
It is not typed out here, since it is 5x5.

1-c)

$$z_{k+1} = Az_k + B\Delta u_k$$

$$z_{k+2} = Az_{k+1} + B\Delta u_{k+1} = A(Az_k + B\Delta u_k) + B\Delta u_{k+1} = A^2z_k + AB\Delta u_k + B\Delta u_{k+1}$$

$$z_{k+3} = Az_{k+2} + B\Delta u_{k+2} = A(A^2z_k + AB\Delta u_k + B\Delta u_{k+1}) + B\Delta u_{k+2} = A^3z_k + A^2B\Delta u_k + AB\Delta u_{k+1} + B\Delta u_{k+2}$$

Starting from  $z_0$ :

$$z_k = A^k z_0 + \sum_{i=0}^{k-1} A^{k-1-i} B\Delta u_i$$

Setting up large QP matrices

$$U = [u_0, u_1, \dots, u_{k-1}]$$

$$Z = [z_0; z_1, \dots, z_k] = [A; A^2; \dots, A^k] * z_0 + [B, 0, \dots, 0; AB, B, 0, \dots, 0; \dots, A^{n-1}B, A^{n-2}B, \dots, B] * U = A_{stack} z_0 + B_{stack} U$$

Substituting into cost:

$$Q = \text{diag}(0, 0, 1, 0, 0), \quad R = 1$$

$$Q_{blk} = \text{blkdiag}(Q, Q, Q, \dots, Q, P)$$

$$R_{blk} = I(N \times N) * R$$

$$J = Z^T Q_{blk} Z + U^T R U = (A_{stack} z_0 + B_{stack} U)^T Q_{blk} (A_{stack} z_0 + B_{stack} U) + U^T R_{blk} U$$

$$J = (z_0^T A_{stack}^T + U^T B_{stack}^T) Q_{blk} (A_{stack} z_0 + B_{stack} U) + U^T R_{blk} U$$

$$J = z_0^T A_{stack}^T Q_{blk} A_{stack} z_0 + U^T B_{stack}^T Q_{blk} B_{stack} U + z_0^T A_{stack}^T Q_{blk} B_{stack} U + U^T B_{stack}^T Q_{blk} A_{stack} z_0 + U^T R_{blk} U$$

Some terms are the same and some terms are constant, which can be removed. Organizing better and dividing by 2:

$$J = (U^T B_{stack}^T Q_{blk} B_{stack} U + U^T R_{blk} U) + 2(z_0^T A_{stack}^T Q_{blk} B_{stack} U) =$$

$$J = U^T (B_{stack}^T Q_{blk} B_{stack} + R_{blk}) U / 2 + (z_0^T A_{stack}^T Q_{blk} B_{stack}) U$$

$$H = (B_{stack}^T Q_{blk} B_{stack} + R_{blk}), \quad q^T = (z_0^T A_{stack}^T Q_{blk} B_{stack})$$

$$J = U^T H U / 2 + q^T U$$

Now setting the constraints on state:

$$GU \leq W + T z_0$$

$$Z \geq A_{stack} z_0 + B_{stack} U \rightarrow B_{stack} U \leq Z + (-A_{stack} z_0)$$

Stacking multiple layers to allow for constraints in both directions and on U:

$$[B_{stack} U \leq Z + (-A_{stack} z_0); B_{stack} U \leq Z + (-A_{stack} z_0); IU \leq Z + (-A_{stack} z_0); IU \leq Z + (-A_{stack} z_0)]$$

$$GU \leq W + T z_0$$

$$G = [B_{stack}; -B_{stack}, I, -I], \quad W = [x_{max}; x_{min}; u_{max}; u_{min}], \quad T = [-A_{stack}; A_{stack}; 0; 0]$$

1-d)

This was implemented in the code shown below:

Defining variables Code:

```
N = 15
Q_mpc = np.diag([0., 0., 1., 0., 0.])
R_mpc = np.array([[1.]])
Pdxe = solve_discrete_are(A[:3,:3], B[:3], np.diag([0.,0.,1.]), R_mpc)
P_mpc = linalg.block_diag(Pdxe, np.zeros((2,2)))

lN = 1e6
xlim = {"max": np.array([lN, lN, lN, u_lim, 0.2]),
        "min": np.array([-lN,-lN,-lN, -u_lim, -0.2])}
ulim = {"max": lN, "min": -lN}

H, L, G, W, T, IMPC = form_qp(A, B, Q_mpc, R_mpc, P_mpc, xlim, ulim, N)
```

QP formulation Code:

```
def form_qp(A, B, Q, R, P, xlim, ulim, N):
    n, m_ = A.shape[0], B.shape[1]

    Psi = np.zeros((n*N, n))
    Gamma = np.zeros((n*N, m_*N))
    Apow = A.copy()
    for i in range(N):
        Psi[i*n:(i+1)*n] = Apow
        Apow = A @ Apow
    for i in range(N):
        for j in range(i+1):
```

```

        Gamma[i*n:(i+1)*n, j*m_:(j+1)*m_] = np.linalg.matrix_power(A, i-j) @
B

# Cost
Qbar = linalg.block_diag(*([Q]*(N-1) + [P]))
Rbar = np.kron(np.eye(N), R)
H = Gamma.T @ Qbar @ Gamma + Rbar
H = (H + H.T) / 2
L = Gamma.T @ Qbar @ Psi

# Constraints
xmax = np.tile(xlim["max"], N)
xmin = np.tile(xlim["min"], N)
umax = np.tile([ulim["max"]], N)
umin = np.tile([ulim["min"]], N)
Iu    = np.eye(m_ * N)
Zu    = np.zeros((m_*N, n))

G = np.vstack([ Gamma, -Gamma,  Iu, -Iu])
W = np.concatenate([xmax, -xmin, umax, -umin])
T = np.vstack([-Psi,  Psi,  Zu,  Zu])

IMPC = np.zeros((m_, m_*N))
IMPC[0, 0] = 1.0
return H, L, G, W, T, IMPC

```

1-e)

The QP problem is solved with the following code:

```

def run_mpc(mp=1.0, kp=1.0, u_lim=0.2):
    N      = 15
    Q_mpc = np.diag([0., 0., 1., 0., 0.])
    R_mpc = np.array([[1.]])
    Pdx = solve_discrete_are(A[:3,:3], B[:3], np.diag([0.,0.,1.]), R_mpc)
    P_mpc = linalg.block_diag(Pdx, np.zeros((2,2)))

    lN    = 1e6
    xlim = {"max": np.array([lN, lN, lN,  u_lim,  0.2]),
            "min": np.array([-lN,-lN,-lN, -u_lim, -0.2])}

```

```

ulim = {"max": lN, "min": -lN}

H, L, G, W, T, IMPC = form_qp(A, B, Q_mpc, R_mpc, P_mpc, xlim, ulim, N)

rng = np.random.default_rng(2)
vals = rng.uniform(-0.19, 0.19, 7).tolist()
holds = rng.integers(25, 60, 7).tolist()
vals.insert(4, -0.25); holds.insert(4, 35) # infeasible command in the
middle
refs_holds = list(zip(vals, holds))

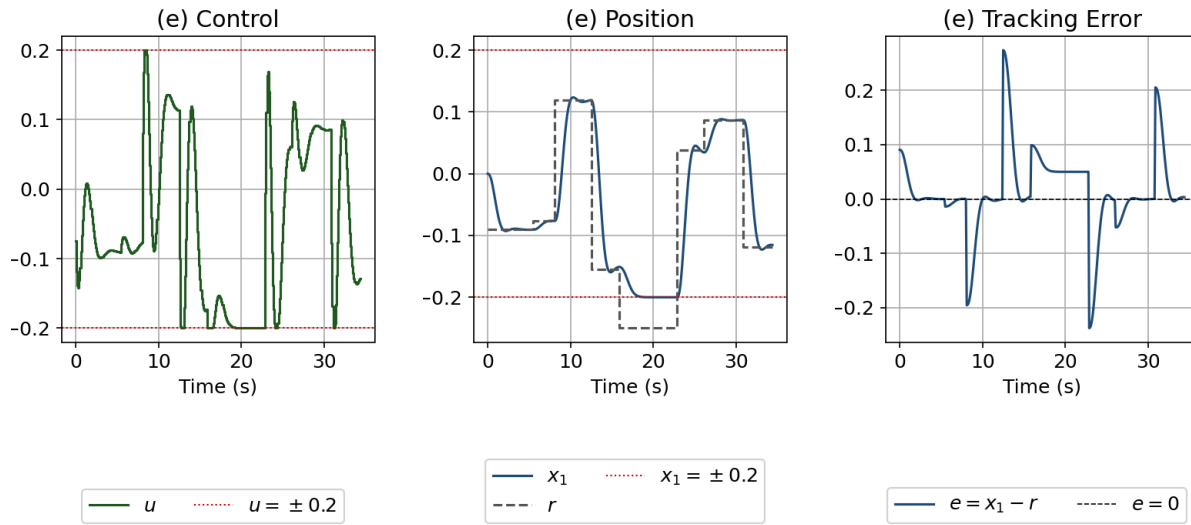
x_c, u_c, dxprev, t_now = np.zeros(2), 0.0, np.zeros(2), 0.0
t_h, x1_h, u_h, r_h = [], [], [], []

for r, hold in refs_holds:
    for _ in range(hold):
        z = np.array([dxprev[0], dxprev[1], x_c[0]-r, u_c, x_c[0]])
        q = L @ z
        rhs = W + T @ z
        res = minimize(
            fun=lambda U: 0.5*(U@H@U) + q@U,
            x0=np.zeros(H.shape[0]),
            jac=lambda U: H@U + q,
            method='SLSQP',
            constraints={'type':'ineq', 'fun':lambda U: rhs-G@U, 'jac':lambda
U:-G},
            options={'ftol':1e-8, 'maxiter':500, 'disp':False}
        )
        u_c += float(np.squeeze(IMPC @ res.x))
        sol = solve_ivp(lambda t,x: msd(t,x,u_c,mp,kp),
                        [t_now, t_now+Ts], x_c, rtol=1e-6, atol=1e-8)
        x_new = sol.y[:,-1]
        dxprev = x_new - x_c
        t_h.append(t_now); x1_h.append(x_c[0])
        u_h.append(u_c); r_h.append(r)
        x_c = x_new; t_now += Ts

t_h.append(t_now); x1_h.append(x_c[0])
u_h.append(u_c); r_h.append(refs_holds[-1][0])
return tuple(map(np.array, (t_h, x1_h, u_h, r_h)))

```

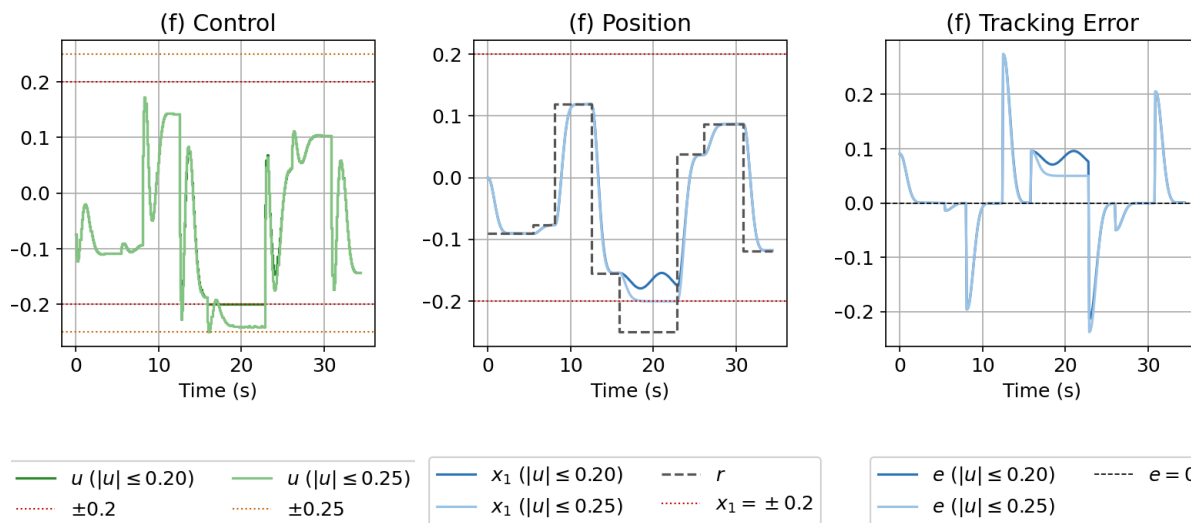
Leading to these plots:



The controller creates fairly smooth trajectories between the desired references which obey the dynamics of the plant. When the control input is infeasible, the MPC controller attempts to get as close as the preset bounds allow for in order to minimize the error, but do not allow it to cross the bounds. The control effort varies between the preset bounds as well, but does not go over the actuation limits.

1-f)

Even when setting the nominal model to be different from the true dynamics model, the MPC controller performs fairly well, but did struggle towards the constraint limits, with the lower actuation limit controller being unable to properly saturate the state constraints. However, increasing the actuation limits did allow the controller to hit the state constraints, where it did not surpass the preset state constraints. In the tracking error and control effort graphs, it can be seen where the lower actuation controller had higher error because the controller effort saturated and did not allow for tracking closer to the state limits.



## Problem 2:

### 2-a)

It was implemented in the code, following the equations given for the dynamics. The skew symmetric matrix and continuous time dynamics were used to compute the discrete time dynamics.

### Code:

```
def skew(w):
    return np.array([[ 0,    -w[2],   w[1]],
                     [ w[2],   0,    -w[0]],
                     [-w[1],   w[0],   0    ]])

def fc(x, u):
    """Continuous-time spacecraft attitude dynamics."""
    w = x[:3]; phi, theta = x[3], x[4]
    wdot = Jinv @ (-skew(w) @ J @ w + u)
    S = np.array([
        [1, np.sin(phi)*np.tan(theta), np.cos(phi)*np.tan(theta)],
        [0, np.cos(phi),                -np.sin(phi)                ],
        [0, np.sin(phi)/np.cos(theta), np.cos(phi)/np.cos(theta)],
    ])
    return np.concatenate([wdot, S @ w])

def fd(x, u):
    """Explicit Euler discretization:  $x_{k+1} = x_k + Ts * fc(x_k, u_k)$ ."""
    return x + Ts * fc(x, u)
```

### 2-b)

$$Q = \text{diag}(100, 100, 100, 10, 10, 10) \rightarrow Q_{blk} = \text{blkdiag}(Q, Q, \dots, Q, P)$$

$$R = \text{diag}(0.1, 0.1, 0.1) \rightarrow R_{blk} = \text{blkdiag}(R, R, \dots, R)$$

P was calculated by linearization around the given operating point:

### Code:

```
def linearize(eps=1e-5):
    """Numerical Jacobians of fd at x=0, u=0."""
    x0, u0 = np.zeros(nx), np.zeros(nu)
    Ad = np.zeros((nx, nx))
    Bd = np.zeros((nx, nu))
    f0 = fd(x0, u0)
    for i in range(nx):
        dx = np.zeros(nx); dx[i] = eps
```

```

Ad[:, i] = (fd(x0 + dx, u0) - f0) / eps
for i in range(nu):
    du = np.zeros(nu); du[i] = eps
    Bd[:, i] = (fd(x0, u0 + du) - f0) / eps
return Ad, Bd

```

To write in the quadratic programming form:

$$X = [x_1, x_2, \dots, x_k], \quad U = [u_0, u_1, \dots, u_{k-1}], \quad r_{full} = [r_0, r_1, \dots, r_{N-1}]$$

Because the Q\_blk matrix has the P term at the end, terms can now be combined:

$$J = (x_n - r)^T P (x_n - r) / 2 + 1/2 * \sum_{k=0}^{N-1} (x_n - r)^T Q (x_n - r) + u_k^T R u_k$$

$$J = ((X - r_{full})^T Q_{blk} (X - r_{full}) + U R_{blk} U) / 2$$

Simplify due to symmetric matrices and distributing transpose:

$$J = (X Q_{blk} X - 2 r_{full}^T Q_{blk} X + r_{full}^T Q_{blk} r_{full}) / 2 + U R_{blk} U$$

Group terms and get rid of constants:

$$z = [X; U]$$

$$J = z^T [Q_{blk}, 0; 0, R_{blk}] z / 2 - [Q_{blk} r; 0] z / 2$$

Which can be written in the correct form with the following substitutions:

$$H = [Q_{blk}, 0; 0, R_{blk}], \quad q^T = -[Q_{blk} r; 0]$$

$$J = z^T H z / 2 - q^T z / 2$$

Constraints to satisfy nonlinear dynamics:

$$x_{k+1} = f(x_k, u_k)$$

$$ceq = [x_1 - f(x_0, u_0); x_2 - f(x_1, u_1) \dots x_N - f(x_{N-1}, u_{N-1})] = 0$$

Can be rewritten in terms of z:

$$ceq = [I, 0] z - [f(x_0, u_0); f(x_1, u_1) \dots f(x_{N-1}, u_{N-1})] = 0$$

2-c)

Input constraints can be added like this:

$$cineq = [0, I; 0, -I] z - [u_{max}; u_{min}] \leq 0$$

And state constraints can be added:

$$cineq = [I, 0; -I, 0] z - [x_{max}; x_{min}] \leq 0$$

The full code for implementing this is shown below:

Code:



```

def build_solver(params):
    X_sym = [ca.MX.sym(f"x{k}", nx) for k in range(N)]
    U_sym = [ca.MX.sym(f"u{k}", nu) for k in range(N)]
    x0_p = ca.MX.sym("x0", nx) # parameter: current state
    r_p = ca.MX.sym("r", nx) # parameter: reference

    # Cost
    cost = 0.5 * (X_sym[-1]-r_p).T @ ca.DM(P) @ (X_sym[-1]-r_p)
    x_prev = x0_p
    for k in range(N):
        cost += 0.5 * ((x_prev-r_p).T @ ca.DM(Q) @ (x_prev-r_p)
                        + U_sym[k].T @ ca.DM(R) @ U_sym[k])
        x_prev = X_sym[k]

    # Dynamics equality constraints
    ceq, x_prev = [], x0_p
    for k in range(N):
        ceq.append(X_sym[k] - fd_ca(x_prev, U_sym[k]))
        x_prev = X_sym[k]

    z = ca.vertcat(*X_sym, *U_sym)
    g = ca.vertcat(*ceq)
    xlb = np.tile(params["xlb"], N)
    xub = np.tile(params["xub"], N)
    ulb = np.tile(params["ulb"], N)
    uub = np.tile(params["uub"], N)

    nlp = {"x": z, "f": cost, "g": g, "p": ca.vertcat(x0_p, r_p)}
    opts = {"ipopt.print_level": 0, "print_time": 0,
            "ipopt.max_iter": 200, "ipopt.tol": 1e-6}
    solver = ca.nlpsol("nmpc", "ipopt", nlp, opts)
    lbz = np.concatenate([xlb, ulb])
    ubz = np.concatenate([xub, uub])
    lbg = np.zeros(nx * N)
    ubg = np.zeros(nx * N)
    return solver, lbz, ubz, lbg, ubg

def solve_nmpc(solver, lbz, ubz, lbg, ubg, x0, r, z_guess):
    """Call the compiled IPOPT solver"""
    sol = solver(x0=z_guess,

```

```

        lbx=lbz, ubx=ubz,
        lbg=lbz, ubg=ubz,
        p=np.concatenate([x0, r]))
z_opt = np.array(sol["x"]).ravel()
X = z_opt[:nx*N].reshape(N, nx)
U = z_opt[nx*N:].reshape(N, nu)
cost = float(sol["f"])
stats = solver.stats()
success = stats["success"]
return U[0], X, U, z_opt, cost, success

def run_sim(params, T_sim=200, x_init=None, r=None, label=""):
    if x_init is None:
        x_init = np.array([0., 0., 0., np.radians(15), np.radians(30),
np.radians(20)])
    if r is None:
        r = np.zeros(nx)

    solver, lbz, ubz, lbg, ubg = build_solver(params)
    steps = int(T_sim / Ts)
    x = x_init.copy()
    z_guess = np.zeros(nx*N + nu*N)

    t_h = np.zeros(steps+1)
    x_h = np.zeros((steps+1, nx))
    u_h = np.zeros((steps, nu))
    cost_h = np.zeros(steps)

    x_h[0] = x
    print(f" {label}: simulating {steps} steps ...")
    for k in range(steps):
        u0, X, U, z_opt, cost, _ = solve_nmpc(solver, lbz, ubz, lbg, ubg, x, r,
z_guess)
        u_h[k] = u0
        cost_h[k] = cost
        X_shift = np.vstack([X[1:], X[-1]])
        U_shift = np.vstack([U[1:], U[-1]])
        z_guess = np.concatenate([X_shift.ravel(), U_shift.ravel()])
        x = fd(x, u0)
        t_h[k+1] = (k+1)*Ts

```

```

x_h[k+1] = x

return t_h, x_h, u_h, cost_h

```

2-d)

The initial constraints and properties were input and the code was run:

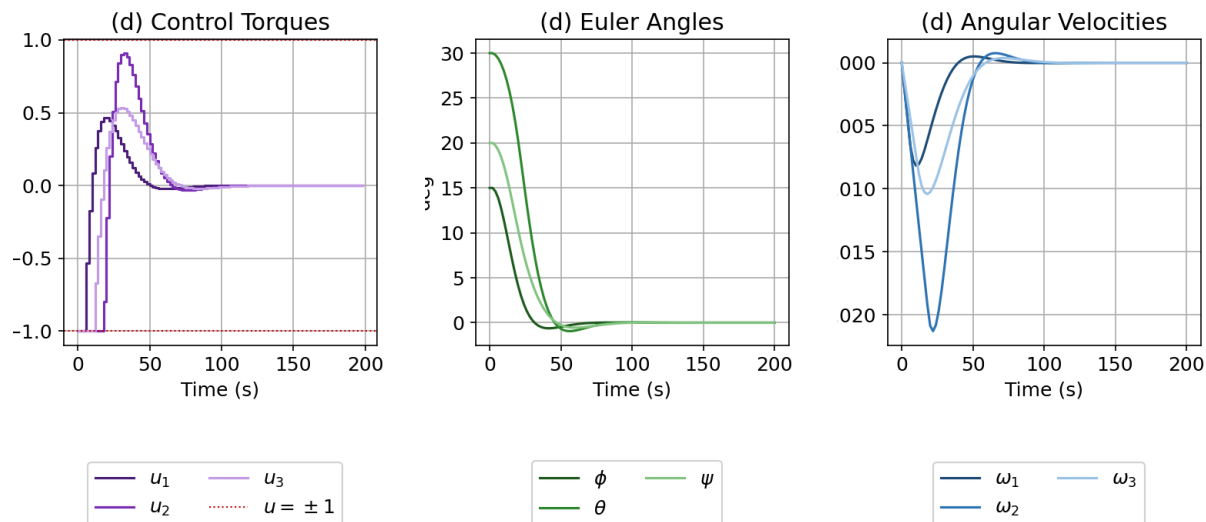
```

lN = 1e3
ulb = np.full(nu, -1.); uub = np.full(nu, 1.)

# Part (d)
params_d = {"xlb": np.full(nx, -lN), "xub": np.full(nx, lN),
            "ulb": ulb, "uub": uub}
print("Running (2d) u")
t_d, x_d, u_d, cost_d = run_sim(params_d, label="(d)")

```

Leading to the following plots:



2-e)

Code for new constraints:

```

# Part (e) Case 1
xmax1 = 10 * np.ones(nx)
xmin1 = -10 * np.array([1,1,1,0.01,0.01,0.01])
params_e1 = {"xlb": xmin1, "xub": xmax1, "ulb": ulb, "uub": uub}
print("Running (e) Case 1")

```

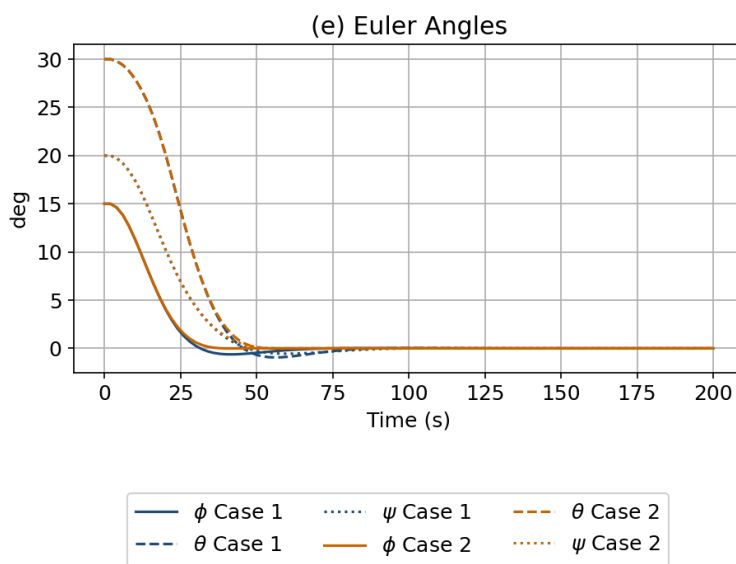
```

t_e1, x_e1, u_e1, cost_e1 = run_sim(params_e1, label="(e) Case 1")

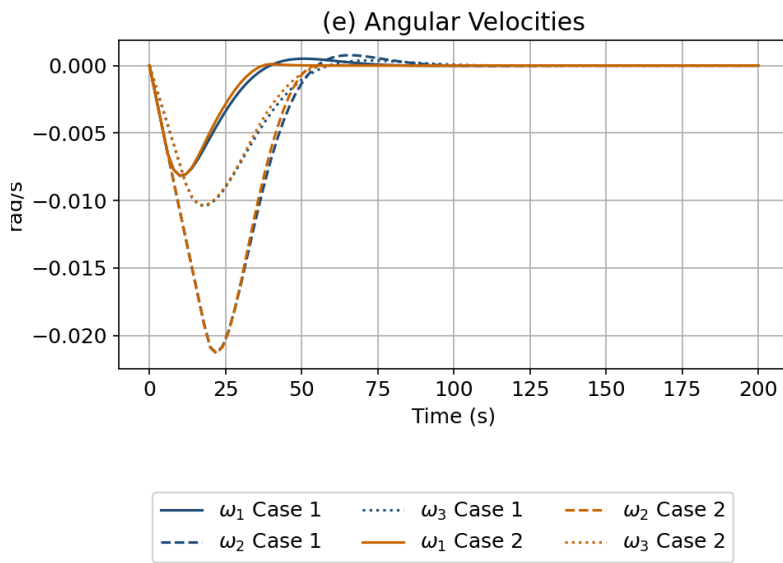
# Part (e) Case 2
xmax2 = np.ones(nx)
xmin2 = -np.array([1,1,1,0,0,0])
params_e2 = {"xlb": xmin2, "xub": xmax2, "ulb": ulb, "uub": uub}
print("Running (e) Case 2")
t_e2, x_e2, u_e2, cost_e2 = run_sim(params_e2, label="(e) Case 2")

```

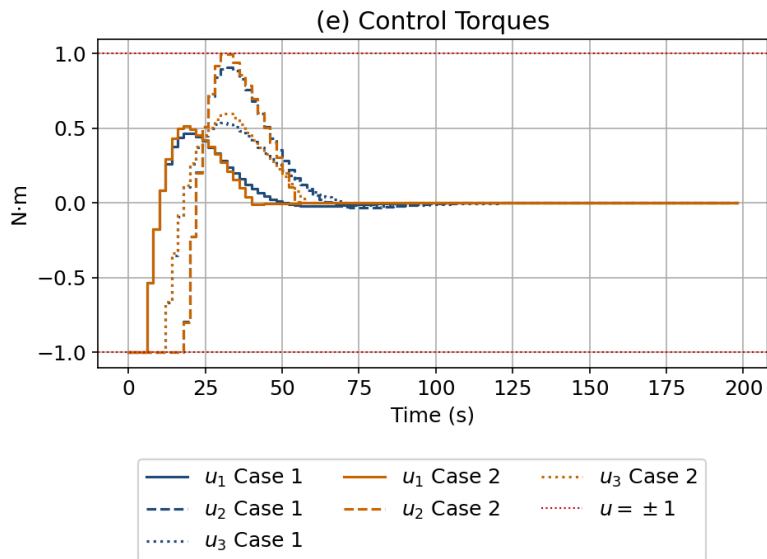
Adding new constraints yields different plots:



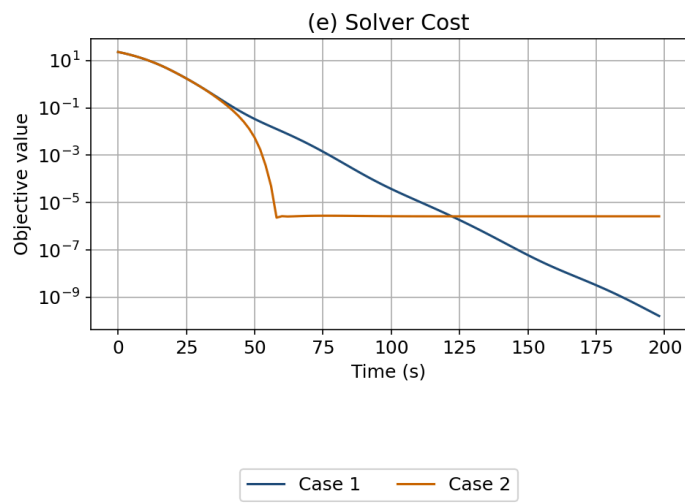
At the start, there is minimal difference between the two cases, but since case 1 is allowed to go below 0, it has some overshoot, where case 2 does not. The overshoot allows it to more slowly adapt to the reference while using less control effort. It has a lower weighting on the state deviations, so this makes sense. In this case, the maneuvering speed actually increased, as it had to avoid a state constraint, but was allowed to use more control effort. This is in contrast to classical control, which tends to overshoot if it wants to correct for something more quickly.



The angular velocities show the same trend, with Case 2 achieving the reference more quickly, while Case 1 had more overshoot. For the most part, the trajectories were unchanged though.



The control torque shows sharper transitions here for case 2, as it needed to quickly cut off control effort to avoid overshooting. Case 1 sloped more gradually to 0, as it did not have such a high cost on state error.



Case 2 cost reduced more quickly, as it used more actuator effort, but then cost flattened and could not go any further. Most likely, this is due to having higher cost coefficients.